

FLUTTER

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Flutter

- 2. Introduction to Dart Language
- 3. Variables and Data Types in Dart
- 4. Functions in Dart

★ 5. What is a Widget?

★ 6. StatelessWidget

★ 7. StatefulWidget

★ 8. MaterialApp and Scaffold

- 9. Text and TextStyle
- 10. Container, Padding and Margin

★ 11. Row and Column

- 12. Stack and Positioned

★ 13. Buttons (ElevatedButton, TextButton, IconButton)

- 14. Images and Assets

★ 15. ListView

- 16. GridView

★ 17. Navigation Between Screens

- 18. Passing Data Between Screens

★ 19. Forms and TextField

★ 20. State Management - setState

- 21. State Management - Provider (Basics)

★ 22. Working with APIs - http package and FutureBuilder

- 23. Local Storage - shared_preferences
- 24. Themes and Styling
- 25. Animations (Basics)

★ 1. Introduction to Flutter [IMPORTANT TOPIC]

Explanation:

Flutter is a free tool made by Google to build apps. With one single code, you can make an app that runs on Android, iPhone, Web, and even Desktop.

Flutter uses a programming language called Dart. Everything you see on the screen in a Flutter app — text, buttons, images — is called a 'Widget'.

Key Points:

- One code -> works on many platforms (Android, iOS, Web, Desktop).
- Fast development using 'Hot Reload' (see changes instantly without restarting the app).
- Everything on screen is a Widget.
- Uses the Dart programming language.

Example Code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Hello Flutter!')),
      ),
    );
  }
}
```

Result:

Screen shows: Hello Flutter! (centered on a plain white screen)

 *Note: runApp() is the starting point of every Flutter app — it tells Flutter which widget to show first.*

2. Introduction to Dart Language

Explanation:

Dart is the programming language used to write Flutter apps. If you already know a little bit of Java, JavaScript, or Python, Dart will feel familiar because its syntax is quite simple.

Example Code:

```
void main() {  
  print('Hello from Dart');  
  int a = 5;  
  int b = 10;  
  print('Sum is: ${a + b}');  
}
```

Result:

```
Hello from Dart  
Sum is: 15
```

 *Note: `\${ }` inside a string is called 'string interpolation' — it lets you insert a variable's value directly into text.*

3. Variables and Data Types in Dart

Explanation:

A variable stores a value that can be used later in the program. Dart is 'type safe', which means every variable has a fixed data type.

Key Points:

- int -> whole numbers, example: 10
- double -> decimal numbers, example: 3.14
- String -> text, example: "Hello"
- bool -> true or false
- var -> Dart automatically decides the data type

Example Code:

```
String name = "Pravendra";  
int age = 21;  
double height = 5.9;  
bool isStudent = true;  
var city = "Delhi"; // Dart detects this is a String  
  
print(name);  
print(age);
```

Result:

```
Pravendra  
21
```

4. Functions in Dart

Explanation:

A function is a reusable block of code that performs a specific task. In Dart, we write the return type, the function name, and the parameters (inputs) it needs.

Example Code:

```
int addNumbers(int a, int b) {  
    return a + b;  
}  
  
void main() {  
    int total = addNumbers(5, 7);  
    print('Total is: $total');  
}
```

Result:

Total is: 12

★ 5. What is a Widget? *[IMPORTANT TOPIC]*

Explanation:

In Flutter, EVERYTHING is a widget — text, buttons, images, even the padding (space) around something is a widget. Widgets are like building blocks (Lego pieces) that we combine together to build the whole screen.

There are two main types of widgets: StatelessWidget and StatefulWidget.

Key Points:

- StatelessWidget -> does not change once it is drawn (example: a simple label).
- StatefulWidget -> can change and update itself on the screen (example: a counter, a checkbox).

Example Code:

```
Text('I am a widget')  
Icon(Icons.star)  
Container(color: Colors.blue)
```

Result:

Screen shows: a text label, a star icon, and a blue box – each one is a separate widget.

★ 6. StatelessWidget [IMPORTANT TOPIC]

Explanation:

A StatelessWidget is used for parts of the screen that do NOT change after being drawn once. For example, a logo, a title, or a static label.

Example Code:

```
class WelcomeText extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text(  
      'Welcome to MMOC Teach!',  
      style: TextStyle(fontSize: 22, color: Colors.blue),  
    );  
  }  
}
```

Result:

Screen shows: 'Welcome to MMOC Teach!' in blue, size 22 text – it never changes.



Note: Use StatelessWidget whenever the widget's data will never update while the app is running.

★ 7. StatefulWidget [IMPORTANT TOPIC]

Explanation:

A StatefulWidget is used when a part of the screen NEEDS to change while the app is running — for example, a counter that increases when a button is pressed. It has two parts: the widget itself, and a 'State' class that holds the changing data.

Example Code:

```
class Counter extends StatefulWidget {
  @override
  _CounterState createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;

  void increase() {
    setState(() {
      count = count + 1;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $count'),
        ElevatedButton(onPressed: increase, child: Text('Add')),
      ],
    );
  }
}
```

Result:

Screen shows: 'Count: 0' with an 'Add' button. Every tap increases the number by 1.

 *Note: setState() tells Flutter: 'data has changed, please redraw the screen'.*

★ 8. MaterialApp and Scaffold *[IMPORTANT TOPIC]*

Explanation:

MaterialApp is the outer wrapper of almost every Flutter app — it sets up themes, colors, and navigation. Scaffold gives the basic page structure: an app bar (top header), a body (main content), and space for buttons like a floating action button.

Example Code:

```
MaterialApp(  
  home: Scaffold(  
    appBar: AppBar(title: Text('My App')),  
    body: Center(child: Text('Welcome!')),  
    floatingActionButton: FloatingActionButton(  
      onPressed: () {},  
      child: Icon(Icons.add),  
    ),  
  ),  
)
```

Result:

Screen shows: a top bar titled 'My App', 'Welcome!' text in the middle, and a round '+' button at the bottom right.

9. Text and TextStyle

Explanation:

The Text widget is used to display words on the screen. We can change its look — size, color, boldness — using TextStyle.

Example Code:

```
Text(  
  'Hello Flutter',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.red,  
  ),  
)
```

Result:

Screen shows: 'Hello Flutter' in bold, red, size 24 text.

10. Container, Padding and Margin

Explanation:

A Container is like a box that can hold one widget inside it. It can have a color, size, border, padding (space inside) and margin (space outside).

Example Code:

```
Container(  
  padding: EdgeInsets.all(16),  
  margin: EdgeInsets.all(10),  
  color: Colors.yellow,  
  child: Text('Inside a box'),  
)
```

Result:

Screen shows: a yellow box with space around the text 'Inside a box' and space outside the box too.

★ 11. Row and Column *[IMPORTANT TOPIC]*

Explanation:

Row arranges widgets side by side horizontally (left to right). Column arranges widgets one below another, vertically. These two are the most used layout widgets in Flutter.

Example Code:

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
  children: [  
    Icon(Icons.home),  
    Icon(Icons.search),  
    Icon(Icons.person),  
  ],  
)
```

Result:

Screen shows: three icons (home, search, person) placed next to each other, evenly spaced.

 *Note: mainAxisAlignment controls spacing along the main direction (horizontal for Row, vertical for Column).*

12. Stack and Positioned

Explanation:

A Stack places widgets on top of each other, like layers. Positioned is used inside a Stack to place a widget at an exact place (top, left, right, bottom).

Example Code:

```
Stack(  
  children: [  
    Image.asset('background.png'),  
    Positioned(  
      bottom: 10,  
      right: 10,  
      child: Text('Logo', style: TextStyle(color: Colors.white)),  
    ),  
  ],  
)
```

Result:

Screen shows: a background image with the word 'Logo' placed on top, near the bottom-right corner.

★ 13. Buttons (ElevatedButton, TextButton, IconButton) *[IMPORTANT TOPIC]*

Explanation:

Flutter provides ready-made buttons. ElevatedButton is a raised button with a shadow. TextButton is a flat button with no background. IconButton is a button that only shows an icon (like a heart or a back arrow).

Example Code:

```
ElevatedButton(  
  onPressed: () {  
    print('Button pressed!');  
  },  
  child: Text('Click Me'),  
)
```

Result:

Screen shows: a raised button labeled 'Click Me'. On tap, the console prints: Button pressed!

 *Note: onPressed is the function that runs when the user taps the button.*

14. Images and Assets

Explanation:

Flutter can show images from the internet (network) or from files saved inside the project (assets). Local images must first be added inside the pubspec.yaml file before they can be used.

Example Code:

```
// pubspec.yaml
// flutter:
//   assets:
//     - images/logo.png

Image.asset('images/logo.png')
Image.network('https://example.com/photo.png')
```

Result:

Screen shows: the logo image from your project folder, and a photo loaded from the internet.

★ 15. ListView [IMPORTANT TOPIC]

Explanation:

ListView is used to show a scrollable list of items — for example, a list of contacts, messages, or products. It is one of the most commonly used widgets in real apps.

Example Code:

```
ListView(  
  children: [  
    ListTile(title: Text('Apple')),  
    ListTile(title: Text('Banana')),  
    ListTile(title: Text('Mango')),  
  ],  
)
```

Result:

Screen shows: a vertically scrollable list with three rows – Apple, Banana, Mango.

 *Note: For long or dynamic lists, use `ListView.builder()` — it creates items only when they are visible, which makes it faster.*

16. GridView

Explanation:

GridView shows items in a grid format (rows and columns), like a photo gallery. It is similar to ListView but arranges items in a box-like grid instead of a single line.

Example Code:

```
GridView.count(  
  crossAxisCount: 2,  
  children: [  
    Container(color: Colors.red),  
    Container(color: Colors.blue),  
    Container(color: Colors.green),  
    Container(color: Colors.orange),  
  ],  
)
```

Result:

Screen shows: 4 colored boxes arranged in a 2-column grid.

★ 17. Navigation Between Screens *[IMPORTANT TOPIC]*

Explanation:

Most apps have more than one screen. Navigator is used to move from one screen (page) to another — for example, going from a login screen to a home screen.

Example Code:

```
// Go to a new screen
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen()),
);

// Go back to the previous screen
Navigator.pop(context);
```

Result:

Screen shows: the app slides to 'SecondScreen'. Pressing back returns to the first screen.

18. Passing Data Between Screens

Explanation:

Sometimes we need to send information (like a username) from one screen to another. This is done by passing values through the constructor of the next screen's widget.

Example Code:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => ProfileScreen(username: 'Pravendra'),
  ),
);

class ProfileScreen extends StatelessWidget {
  final String username;
  ProfileScreen({required this.username});

  @override
  Widget build(BuildContext context) {
    return Scaffold(body: Center(child: Text('Hello $username')));
  }
}
```

Result:

Screen shows: 'Hello Pravendra' on the ProfileScreen page.

★ 19. Forms and TextField *[IMPORTANT TOPIC]*

Explanation:

TextField lets the user type text into the app, like entering a name or password. A Form widget groups multiple input fields together and helps validate them (check if they are filled correctly) before submitting.

Example Code:

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Enter your name',  
    border: OutlineInputBorder(),  
  ),  
  onChanged: (value) {  
    print('User typed: $value');  
  },  
)
```

Result:

Screen shows: an input box labeled 'Enter your name'. Whatever the user types is printed live in the console.



Note: Use a TextEditingController when you need to read or clear the text field's value from your code.

★ 20. State Management - setState [IMPORTANT TOPIC]

Explanation:

State management means controlling how data changes and how the screen updates when that data changes. The simplest way in Flutter is `setState()`, which is good for small apps or a single screen.

Example Code:

```
int likes = 0;

void addLike() {
  setState(() {
    likes++;
  });
}
```

Result:

Screen shows: the like count increases by 1 every time `addLike()` runs, and the screen refreshes instantly.

💡 *Note: `setState()` only works inside a `StatefulWidget`'s `State` class.*

21. State Management - Provider (Basics)

Explanation:

As an app grows bigger, using only `setState()` becomes messy. Provider is a popular package that helps share and manage data across many screens easily, without passing it manually everywhere.

Example Code:

```
class CounterProvider extends ChangeNotifier {  
  int count = 0;  
  
  void increment() {  
    count++;  
    notifyListeners(); // tells all screens using this data to update  
  }  
}
```

Result:

(No screen output here – this class simply manages shared data used across multiple screens.)

 *Note: `notifyListeners()` is like `setState()`, but it can update **MANY** screens at once, not just one.*

★ 22. Working with APIs - http package and FutureBuilder

[IMPORTANT TOPIC]

Explanation:

Real apps usually fetch data from the internet (like weather, news, or user data) using APIs. The 'http' package is used to send a request to a server, and 'FutureBuilder' is used to show that data on the screen once it arrives.

Example Code:

```
import 'package:http/http.dart' as http;

Future<String> fetchData() async {
  var response = await http.get(
    Uri.parse('https://api.example.com/data'),
  );
  return response.body;
}
```

Result:

(Returns the data from the server once the internet request finishes.)

 *Note: async and await are used because fetching data from the internet takes time — the app should not freeze while waiting.*

23. Local Storage - shared_preferences

Explanation:

Sometimes we want to save small pieces of data on the phone itself, like a login status or app settings, even after the app is closed. The 'shared_preferences' package is used for this kind of simple, permanent storage.

Example Code:

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> saveName(String name) async {
  SharedPreferences prefs = await SharedPreferences.getInstance();
  prefs.setString('username', name);
}
```

Result:

(The value 'name' is saved on the device and stays there even after the app restarts.)

24. Themes and Styling

Explanation:

A Theme lets you set common colors, fonts, and styles once for the whole app, instead of repeating the same style on every screen. This keeps the app looking consistent everywhere.

Example Code:

```
MaterialApp(  
  theme: ThemeData(  
    primaryColor: Colors.deepPurple,  
    scaffoldBackgroundColor: Colors.white,  
  ),  
  home: HomeScreen(),  
)
```

Result:

Screen shows: the whole app now uses deep purple as its main color and a white background by default.

25. Animations (Basics)

Explanation:

Animations make an app feel smooth and alive — like a button that grows, or a box that slides in. Flutter has many ready-made animated widgets, and 'AnimatedContainer' is one of the easiest ones to start with.

Example Code:

```
AnimatedContainer(  
  duration: Duration(seconds: 1),  
  width: isBig ? 200 : 100,  
  height: isBig ? 200 : 100,  
  color: Colors.blue,  
)
```

Result:

Screen shows: a blue box that smoothly grows or shrinks over 1 second whenever 'isBig' changes.

 *Note: You only need to change the value (like width) — Flutter automatically animates the change for you.*